

Open-Domain Question Answering Wrap-Up Report

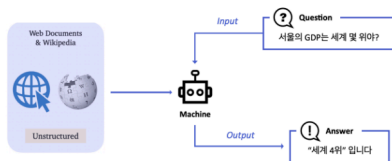
1. 프로젝트 개요 및 구성

1-1. 개요

작성자	nlp-03
기간	2025.12.03 10:00 ~ 2025.12.11 19:00
최종 리더보드	

Linking MRC and Retrieval: Open-domain Question Answering (ODQA)

ODQA: 질문이 따로 주어지지 않음. 방대한 World Knowledge에 기반해서 질의응답



본 프로젝트는 사용자의 질문만으로 적절한 정보를 찾아 답을 생성하는 **Open-Domain Question Answering(ODQA)** 시스템을 구현하는 것을 목표로 한다. 이를 위해 시스템은 두 단계로 구성된다.

1. **Retriever** 단계에서 질문과 관련된 문서를 효율적으로 탐색하고,
2. **Reader** 단계에서 선택된 문서를 기반으로 정답을 추론하거나 생성한다.

최종 모델은 사용자가 입력한 Query에 대해 적절한 문자열 형태의 답을 출력하도록 설계한다. 평가 기준은 **Exact Match(EM)** 와 **F1 Score**를 사용하며, 랭킹은 EM을 기반으로 산정된다. EM은 정답과 완전히 일치할 때만 점수를 부여하고, F1은 일부분이 맞는 경우에도 부분 점수를 허용하는 참고 지표로 활용된다.

The definition of true positive (TP), true negative (TN), false positive (FP), false negative (FN)

	Tokens in Reference	Tokens Not in Reference
tokens in candidate	TP	FP
tokens not in candidate	FN	TN

$$\text{Precision} = \frac{\text{num}(\text{same_token})}{\text{num}(\text{pred_tokens})}$$

$$\text{Recall} = \frac{\text{num}(\text{same_token})}{\text{num}(\text{ground_tokens})}$$

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

1-2. 개발 환경

개발환경	Ubuntu / Python 3.10 / PyTorch / Transformers / scikit-learn / wandb
GPU	Tesla V100-SXM2-32GB
관리	VSCoDe Remote SSH, W&B 실험 로그 관리, VPN 사용
모델 허브	Hugging Face Transformers

1-3. 프로젝트 팀 구성 및 역할

팀원	역할
김윤희	데이터 EDA, hybrid retrieval 실험 및 구현, 앙상블, retrieval 및 reader fine-tuning
박서진	Dense retrieval 실험 및 모델 선정, retrieval 파인튜닝, 데이터 전처리
곽나영	데이터 전처리, Hybrid Retrieval 실험 및 구현, Retrieval Fine-tuning
김이슬	KorQuAD2 데이터 전처리 및 증강, ElasticSearch Retrieval 구현 및 성능 실험
우혜진	Retrieval(EM25) 구현 및 실험, Reader 모델 개선(Negative Passage, Curriculum Learning, Qwen3)
최준호	Curriculum Learning 난이도 지표 구성, 앙상블 구현

2. 프로젝트 수행 절차 및 방법

2-1. 데이터 및 베이스라인 코드 분석

2-1-1. Corpus 및 정답 Passage 특징

특징	수치	의미 및 해석
총 Corpus Passage 수	60,613개	방대한 검색 공간임, Sparse Retrieval이 이 공간에서 정답 문서를 찾아야함.
고유 문서 제목 수	31,755개	하나의 Wikipedia 문서가 여러 개의 Passage로 나뉘어 있음을 의미
정답 문서 ID 수	3,340개	전체 Corpus 중 약 5.5%의 Passage만 정답을 포함
최다 사용 문서 (ID: 5284)	4회	문서 재사용 빈도가 매우 낮음, 대부분의 질문은 고유한 Passage에 연결, 4회는 특정 주제에 대한 질문이 몇 개 더 있을 뿐, 특정 문서에 과도하게 의존하지 않음

전체 6만여 개의 문서 중 실제 정답을 포함하는 Passage의 비율이 5.5%에 불과하고, 문서 재사용 빈도도 낮다. 이는 모델이 특정 문서에 과적합될 위험이 낮으나, 동시에 Retriever가 매우 넓고 희소한 검색 공간에서 정답 Passage를 찾아내야 함을 의미한다. 따라서 Retriever의 강력한 일반화 성능(Robustness) 확보가 시스템 성능 극대화의 핵심임을 진단했다.

2-1-2. Reader 입력 데이터 길이 분석



특징	수치	의미 및 해석
질문 길이 (평균)	16.3 토큰	질문이 비교적 짧고 간결 (BERT/RoBERTa 계열 모델이 질문을 충분히 인코딩 가능)
Context 길이 (평균)	495.3 토큰	매우 길다. 문맥의 평균 길이가 max_seq_length (384)를 초과.
Context 길이 (75%)	592.0 토큰	75%의 Context가 384 토큰보다 훨씬 김
Context 길이 (최소)	239.0 토큰	모든 Context가 239 토큰 이상으로, 대부분 긴 문서에서 발췌되었다.

Reader 모델의 최대 입력 길이(max_seq_length)가 384로 설정될 경우, 대다수의 Context에서 후반부가 잘려 나가 정답 위치가 손실될 가능성이 높다. 정답이 잘린 Context에서는 Reader가 답변을 찾을 수 없으므로, Context 정보 손실을 최소화하기 위해 max_seq_length를 상향 조정(예: 512 이상)할 필요성을 확인하였다.

2-1-3. 답변 길이 특성 분석 (Answer Span)

특징	수치	의미 및 해석
길이 평균	3.5 토큰	답변 길이가 매우 짧고 간결, 대부분 단어/구 단위의 명확한 정답임
길이 최대	49 토큰	극히 일부의 예외적인 긴 답변이 존재
30토큰 초과	4개 (총 3952개 중)	전체 의 약 0.1%만이 긴 답변을 가짐

답변 길이의 99% 이상이 30 토큰 이내에 분포되어 있으므로, 현재 max_answer_length=30 설정은 대부분의 케이스를 커버하는 데 적절하다. 다만, 시스템의 안정성을 극대화하고 극소수 예외 케이스(최대 49 토큰)를 포괄하기 위해, 35 토큰으로의 소폭 상향 조정을 고려하여 Reader 모델의 성능 범위를 넓힐 수 있음을 확인하였다.

2-2. 데이터 가공

2-2-1 . passage 처리

기존의 Wikipedia JSON 파일은 하나의 문서가 통째로 하나의 입력 단위로 구성되어 있다. 그러나 이러한 형태는 질문과 직접적으로 연관된 특정 문맥을 찾기에 어렵지 않을까 하는 생각을 했다. 따라서 Retrieval 단계에서 문서를 chunking된 passage 단위로 분할하여 입력으로 사용하면, 질문과 관련성이 높은 정보를 보다 쉽고 정확하게 탐색할 수 있을 것이라는 가정에서 접근하였다.

구체적으로는, 널리 사용되는 256 토큰 크기의 passage를 기본 단위로 문서를 분할하였으며, passage 경계에서 발생할 수 있는 문맥 손실을 최소화하기 위해 stride 128을 적용하여 인접한 passage 간 문맥이 일부 겹치도록 구성하였다.

2-2-2. Question 전처리

질문 전체 문장을 사용하는 방식과 비교하여, MeCab 형태소 분석기를 활용해 질문에서 핵심 명사만을 추출한 뒤 이를 질의 앞에 결합하는

방식도 함께 실험하였다. 해당 접근은 질문의 핵심 키워드를 강조하여 검색 결과의 초기 관련성을 높이고자 한 시도였다.

그러나 형태소 기반 질의는 명사 중심으로 구성되면서 조사, 어미, 시간적·관계적 의미를 포함하는 표현들이 제거되어 질문의 의도와 문맥 정보를 충분히 반영하지 못하는 한계가 관찰되었다. 특히 ‘처음으로’, ‘어떤 관계에서’, ‘어떤 기준으로’와 같이 질문의 핵심 조건을 형성하는 단어들이 누락되면서, 정답 문맥과의 의미적 일치도가 낮아지는 문제가 발생하였다.

그 결과, 형태소 기반 질의는 특정 키워드 중심의 질문에서는 일부 효과를 보였으나, 문맥적 해석이 중요한 질문의 경우 오히려 성능이 하락하는 경향을 보여 주요 질의 방식으로는 적합하지 않다고 판단하였다.

2-2-3. Multi-Query

단일 질문만을 사용한 문서 검색은 질문 표현 방식에 따라 관련 문서가 충분히 검색되지 않는 한계가 있다. 이를 보완하기 위해 본 실험에서는 LLM을 활용한 Multi-Query 기반 질의 확장 방식을 시도하였다.

구체적으로, 입력 질문을 OpenAI의 경량 LLM 모델인 o4-mini에 입력하여 의미는 동일하되 표현이 다른 유사 질문 5개를 생성하고, 원 질문과 함께 Retriever 단계에 제공하였다. 해당 모델은 다수의 질의를 빠르고 효율적으로 생성할 수 있다는 점에서 실험 환경에 적합하다고 판단하였다.

그러나 실험 결과, 생성된 질의들 간의 의미 중복도가 높거나 질문의 핵심 조건이 일부 변형되는 경우가 관찰되었으며, 이로 인해 검색 결과에 노이즈 문서가 포함되는 문제가 발생하였다. 또한 질의 수 증가에 따른 계산 비용 대비 EM 및 F1 지표에서 일관된 성능 향상을 확인하지 못하여, 해당 방식은 최종 질의 전략으로는 적합하지 않다고 판단하였다.

2-3. Retriever 모델 개선

질의응답(ODQA) 시스템의 검색 정확도와 안정성을 극대화하기 위해, 키워드 기반의 Sparse Embedding과 의미 기반의 Dense Embedding의 장점을 모두 활용하는 하이브리드 검색(Hybrid Retrieval) 전략을 핵심으로 채택하고, 이를 다단계 파이프라인으로 정교화했다.

2-3-1. Sparse Embedding 개선

먼저 Sparse Embedding의 성능을 더 향상 시키기 위해 Baseline의 TF-IDF보다 조금 더 업그레이드된 BM25를 테스트 해봤다. BM25는 문서 길이와 불용어의 영향력이 줄이고, 또 특정 단어의 등장 횟수가 과도하게 반영되지 않도록 조정하는 장점이 있다.

방법	public	private	model 및 상태
Baseline	34.17	47.63	대회 제공
BM25	44.17	57.34	Sparse Embedding 개선

2-3-2. dense model 개선

1. dense retrieval 모델 성능 비교

키워드 기반의 Sparse Retrieval 방식의 한계를 보완하고 의미 기반 검색 성능을 확보하기 위해 Dense retrieval 모델을 도입하였다. Dense retrieval 비교 대상 모델로는 multilingual-e5, Qwen embedding, BGE-M3 계열 임베딩 모델을 Dense Retriever 후보로 선정하였다. 각 모델에 대해 동일한 데이터셋과 동일한 retrieval 설정(top-k 기준)을 적용하여 Recall@k 지표를 중심으로 성능을

평가하였다.

실험 결과, 비교한 모델 중 BGE-M3 모델이 가장 우수한 성능을 보였다. 이에 따라 해당 모델을 최종 Dense Retriever로 선정하였다.

		recall@1	recall@5	recall@10	recall@20
BAAI /bge-m3	train	70.67%	84.82%	89.22%	92.26%
	valid	75.42%	86.67%	90.42%	93.33%
multiling ual-e5- base	train	70.07%	84.69%	88.87%	92.00%
	valid	69.17%	89.17%	92.50%	94.58%
multiling ual-e5- small	train	67.71%	83.38%	87.88%	90.94%
	valid	72.08%	83.33%	86.67%	90.83%
Qwen- Embedd ing-0.6B	train	63.94%	79.12%	83.58%	87.12%
	valid	70.42%	83.75%	86.25%	90.00%

2. Dense retrieval 모델 성능 향상(+ 데이터 전처리)

선정된 BGE-M3 모델의 성능을 추가적으로 향상시키기 위해 질문-문서 간 **relevance**를 학습하는 방식으로 파인튜닝을 수행하였다. 학습 데이터는 **positive-negative pair** 구조로 구성하였으며, 하나의 질문에 대해 정답 문서를 **positive**로, 의미적으로 유사하지만 정답이 아닌 문서를 **negative**로 포함시켰다. 모델 학습에는 **contrastive learning** 방식을 적용하였고, 질문 임베딩과 문서 임베딩 간의 **cosine similarity**를 기반으로 **positive pair**는 가깝게, **negative pair**는 멀어지도록 학습을 유도하였다.

파인튜닝 이후 동일한 평가 환경에서 **Recall@k**를 재측정한 결과, 예상과 달리 성능이 오히려 저하되는 현상을 확인하였다. 이를 통해 제한된 학습 데이터 규모와 **negative sampling** 전략이 Dense Retriever 파인튜닝 성능에 영향을 미칠 수 있음을 확인하였으며, 파인튜닝이 항상 성능 향상으로 이어지지는 않는다는 점을 실험적으로 관찰하였다.

3.concat 앙상블

단일 모델이 놓칠 수 있는 다양한 의미론적 특징을 포착하기 위해, 여러 개의 **pre-trained sentence Transformer** 모델을 활용하는 **embedding concatenation** 앙상블을 사용하였다. DenseREtrievalEnsemble 클래스 내에서 query 와 Passage를 각 모델로 인코딩한 후, 개별 모델의 임베딩 벡터를 축을 따라 이어 붙여 하나의 초고차원 벡터를 생성하였다. 이 벡터에 대해 내적을 계산함으로써 여러 모델의 강점을 결합하여 단일 모델 대비 초기 검색의 변별력을 높였다

2-3-3. hybrid

1.sparse + dense

초기에는 Sparse Embedding과 Dense Embedding를 결합하기 위해 Hybrid Retrieval 방식을 적용하였다. 구체적으로는 BM25 기반 Sparse Retrieval 점수와 Dense Encoder(BAAI/bge-m3) 기반 Dense Retrieval 점수를 각각 계산한 뒤, 두 점수를 가중 합(alpha)으로 결합하여 최종 Relevance score를 계산했다. alpha를 0.1 ~ 0.9까지 0.1 단위로 테스트 해본 결과 BM25가 0.7, bge-m3가 0.3이었을 때의 Recall이 가장 높게 나왔다.

그리고 조금 더 Ranking 보정을 하기 위해 Cross-Encoder 기반 Reranker를 파인튜닝 없이 Pretrained 모델 (Dongjin-kr/ko-reranker)을 추가로 적용하여 1차 Retrieval 모델을 구축하였다.

alpha	0.1	0.2	0.3	0.4	0.5
Recall@5	85.42	91.67	93.33	92.5	91.67
alpha	0.6	0.7	0.8	0.9	-
Recall@5	90.42	90.0	87.08	83.33	

2. reciprocal rank funsion 적용

상호 보완적인 DR과 BM25의 랭킹을 안정적으로 통합하여 문서 순위를 끌어올리는 추가적인 방법을 적용해보았다.

두 검색 엔진에서 각각 K개의 후보 문서를 추출한 후, **reciprocal rank funsion** 기법을 사용하여 두 검색 결과의 순위 정보를 하나의 통합된 순위 리스트로 결합하였다. rrf는 점수 스케일이 상이한 검색 엔진의 결과를 순위 정보만을 활용하여 융합함으로써 가장 안정적인 하이브리드 통합을 제공한다

$$Score(d) = \sum_{r \in Ranks} \frac{1}{K + rank(d)}$$

이 방식은 순위가 높은 문서에 가중치를 부여하고, 두 리스트 모두에서 높은 순위를 얻은 문서에 높은 최종 점수를 준다.

3.rerank 적용

rrf를 거쳐 통합된 Top-K개의 후보군에 대해 Cross-Encoder 를 활용하여 관련도 점수를 극대화하고, 최종적으로 reader 모델에게 최적의 문맥을 전달하는 것을 목표로 Reranking 을 적용하였다. rrf 융합 단계에서 추출된 60 개의 후보 문서 중, top-20개의 문서를 rerank candidate로 설정하였다. query와 passage를 [cls] query [sep] passage [sep] 형태로 단일 입력 시퀀스로 만들어 모델에 투입하였다

이 단일 입력 방식은 BERT 모델의 full attention 메커니즘을 통해 질의와 문서 간의 모든 상호작용이 토큰 레벨에서 정밀하게 분석하여 질문과 문서의 관련성을 정확하게 예측하도록 한다.

Cross-Encoder로 Top-20 리스트를 재순위함으로써, 의미적으로 유사하거나 정답과는 거리가 먼 후보들을 정확하게 순위에서 밀어내고 정답문서를 최종 5 개 내외로 집중시키는 효과를 얻었다. 이 단계에서 검색 정확도의 최종적인 병목 구간을 해결하는 과정을 진행하였다.

2-3-4. 실험 성능 비교

방법	public	private	model 및 상태
Baseline	34.17	47.63	대회 제공
dense rerank	62.50	61.11	cadidate_rerank = 20
dense cocnat	61.67	58.06	"jhgan/ko-sroberta-multitask", "BAAI/bge-m3"
concat + rrf	64.17	57.50	rrf_k = 60

2-4. Reader 모델 개선

2-4-1. 베이스 모델 선택

model	EM	F1
BERT	56.25%	65.7256%
RoBERTa-large	66.6667%	74.1281%
KR-BERT-MRC	55.8333%	67.1966%
Hantaek/klue-roberta-large-korquad-v1-qa-finetuned	68.75%	76.9726%

RoBERTa-large 모델은 BERT 구조를 기반으로 사전학습 전략을 개선하고 모델 규모를 확장한 언어 모델로, 더 많은 파라미터와 대규모 학습 데이터를 활용해 문맥 이해 성능을 향상시켰다. 실제 실험에서도 BERT 대비 EM과 F1 점수가 모두 유의미하게 개선되어, 한국어 MRC 태스크에 적합한 기본 모델로 판단하였다.

KR-BERT-MRC는 한국어 텍스트에 특화된 BERT 기반 기계독해(MRC) 모델로, 한국어 형태와 어휘 특성을 반영한 사전학습이 적용된 모델이다. 본 실험에서는 EM 55.83%, F1 67.20%의 성능을 기록하였으며, 이를 통해 한국어 특화 BERT 계열 모델의 MRC 성능을 비교 기준으로 활용하였다.

Hantaek/klue-roberta-large-korquad-v1-qa-finetuned 모델은 KLUE RoBERTa-large를 기반으로 한국어 질의응답 데이터셋인 KorQuAD v1을 활용해 MRC 태스크에 특화되도록 파인튜닝된 모델이다. 성능 비교 실험 결과, EM 68.75%, F1 76.97%로 전체 모델 중 가장 높은 성능을 기록하였으며, 이는 단순 언어 모델 대비 질문-지문 간 정밀한 의미 매칭 능력이 향상되었음을 의미한다.

이에 따라 한국어 MRC 환경에서 가장 안정적이고 우수한 성능을 보인 Hantaek/klue-roberta-large-korquad-v1-qa-finetuned 모델을 최종 QA 모델로 선정하였다. 더불어 후술될 대부분의 실험은 해당 모델로 진행되었음을 밝힌다.

2-4-2. Negative Passage

Hard negative(정답과 의미적으로 유사하지만 실제로는 정답을 포함하지 않는 문단)를 학습 데이터에 포함시키는 방식은 모델의 문서 판별 능력과 EM 성능을 향상시키는 데 효과적인 기법이다. 실제로 *Dense Passage Retrieval (DPR)*, *REALM*, *ANCE* 등 여러 Retrieval 기반 QA 연구에서도 hard negative passage를 명시적으로 구성하여 학습에 활용하는 것이 핵심 기법으로 널리 사용된다.

Baseline Reader는 학습 시 정답을 포함한 단일 context만을 입력으로 사용하지만, 실제 추론 환경에서는 Retrieval 단계에서 가져온 여러 문서 중 하나에서 정답을 찾아야 한다. 이로 인해 학습-추론 간 설정 불일치가 발생하며, Reader가 유사한 오답 문단에

취약해지는 문제가 나타난다. 이를 완화하기 위해 여러 후보 문단 중 정답을 판별하도록 Negative Passage를 포함한 학습 방식을 적용하였다.

Negative Passage Dataset을 구성할 때는, 학습과 추론 간의 분포 차이를 최소화하기 위해 Retrieval 단계와 최대한 동일한 로직을 사용하는 것을 목표로 하였다. 구체적으로는 BM25와 Dense Retrieval 결과를 결합한 RRF(Reciprocal Rank Fusion) 기반 Hybrid Retrieval을 적용하고, 이후 Cross-Encoder 기반 Reranker를 사용하여 inference 단계와 동일한 방식으로 passage를 재정렬하였다. 이를 통해 실제 추론 시 모델이 접하게 되는 문단 구성과 매우 유사한 환경을 학습 데이터에서도 재현하고자 하였다. 또한 정답 passage의 위치를 랜덤하게 섞어 position bias를 완화함으로써, 모델이 단순한 위치 정보가 아닌 문맥적 근거를 기반으로 답을 찾도록 유도하였다.

이러한 Negative Passage 기반 학습 전략을 적용한 결과, 기존 방식 대비 EM 및 전반적인 성능 향상이 관찰되었다. 특히 hard negative가 포함된 설정에서 모델의 오답 선택 감소와 정답 문단 판별 능력이 향상됨을 확인할 수 있었다.

Idea	Public		Private	
	EM	F1	EM	F1
Negative Passage 미적용	61.25	72.48	57.78	71.71
Negative Passage 적용	64.58	76.09	62.78	77.22

2-4-3. Curriculum Learning

앞서 구성한 Negative Passage Dataset을 난이도에 따라 단계적으로 학습하는 Curriculum Learning 전략을 적용하였다. Curriculum Learning의 핵심 원리는 모델이 상대적으로 쉬운 데이터부터 점진적으로 어려운 데이터로 학습함으로써 안정적인 수렴과 일반화 성능을 동시에 향상시키는 것이다.

passage 개수를 기준으로 Easy(3 passages), Medium(5 passages), Hard(7 passages)로 난이도를 구분하여, 정보량과 distractor 수가 단계적으로 증가하도록 설계하였다. 또한 정답 passage가 항상 동일한 위치에 나타나는 position bias를 방지하기 위해, 각 학습 샘플에서 정답 passage의 위치를 무작위로 배치하였다. 이를 통해 모델이 특정 위치에 의존하지 않고, 문맥 전체를 탐색하며 정답을 식별하도록 유도하였다.

실험 결과, Curriculum Learning을 적용한 설정에서 전반적인 성능 향상이 관찰되었다. 특히 Hard(7 passages) 데이터만으로 단일 단계 학습을 진행한 경우보다, Easy → Medium → Hard 순서로 학습한 경우가 더 우수한 성능을 보였다. 이는 초기 학습 단계에서 상대적으로 단순한 context 구성을 통해 정답 위치 예측과 span 추출에 대한 기본적인 판단 능력을 안정적으로 학습한 후, 정답 위치가 고정되지 않은 상태에서도 점진적으로 distractor가 많은 어려운 환경에 노출됨으로써, 복잡한 문맥 내에서도 정답을 식별할 수 있는 판별 능력이 효과적으로 강화되었기 때문으로 해석했다.

Idea	Public		Private	
	EM	F1	EM	F1
7 passages Only	63.75	76.39	62.50	75.72

curriculum learning (3→5→7)	65.00	77.10	66.67	78.38
--------------------------------	-------	-------	-------	-------

Reader 모델 학습 시, 기존 train dataset을 그대로 활용하여 학습을 진행할 경우 오히려 성능이 저하되는 현상을 여러 차례 관찰하였다. 이는 train dataset이 단일 정답 문서 기반으로 구성되어 있어, 모델이 retrieval 환경에서 요구되는 문서 간 판별 능력보다는 특정 문맥 패턴에 과도하게 적응(overfitting)했기 때문으로 해석하였다. 이러한 성능 저하가 특정 학습 데이터 분포에 대한 과도한 적응에서 비롯되었을 가능성을 고려하여, 다양한 질문 유형과 문맥에 노출함으로써 일반화 성능을 개선할 수 있는지 검증하고자 KorQuAD2 데이터를 Curriculum Learning 단계에 포함하여 학습을 수행하였다. (KorQuAD2 → Easy → Medium → Hard 구조)

그러나 실험 결과 전반적인 성능 향상은 관찰되지 않았으며, 일부 설정에서는 오히려 성능 하락이 나타났다. 이는 KorQuAD2가 단일 문서 기반 QA 데이터라는 특성으로 인해 Negative Passage 환경과 학습 목표 간 불일치가 존재하고, 이미 retrieval-adaptive한 학습을 거친 모델에 대해 추가적인 데이터 분포 혼합이 학습 안정성을 저해했을 가능성이 있기 때문으로 판단하였다. 이러한 이유로 KorQuAD2를 포함한 Curriculum Learning 확장 실험은 추가적인 개선 방향이 명확해지기 전까지 더 이상 진행하지 않았다.

2-4-4. KorQuAD2 데이터

기존 QA 모델로 사용한 Hantaek/klue-roberta-large-korquad-v1-qa-finetuned 모델은 KorQuAD v1 데이터셋을 기반으로 학습된 모델이기 때문에, 보다 다양한 질문 유형과 문맥을 학습시키기 위해 추가적인 데이터 확장이 필요하다고 판단하였다. 이에 따라 동일한 한국어 질의응답 데이터셋 계열인 KorQuAD 2.0 데이터를 추가로 활용하였다.

그러나 KorQuAD 2.0 데이터를 실제로 확인한 결과, 기존에 사용하던 KorQuAD v1 기반 데이터 및 base 데이터와 비교했을 때 JSON 구조가 상이하고, 위키피디아 원문에서 추출된 HTML 태그, 표 구조, 각주, 편집 영역 등 다양한 노이즈 요소가 포함되어 있어 모델 학습 및 추론에 그대로 사용하기에는 적합하지 않은 상태임을 확인하였다.

이에 따라 KorQuAD 2.0 데이터를 기존 QA 파이프라인에 적용하기 위해, 데이터 구조를 SQuAD 형태로 변환하고 학습에 불필요한 노이즈를 제거하는 전처리 과정을 수행하였다. 특히 HTML 태그, 표 형식 문맥, 위키피디아 편집 영역 등을 제거하고, 전처리 이후에도 정답이 문맥 내에 정확히 존재하는 경우만을 선별하여 데이터 품질을 확보하였다. 그 결과, 약 2,000여 개의 유효한 QA 샘플을 확보하여 추가 학습 및 실험에 활용했다.

2-4-5. Qwen3 기반 Reader 확장

최근 ODQA/QA 대회에서는 기존 BERT 계열 span-based reader 단독 구조에서 벗어나, LLM 기반 encoder-decoder 모델을 정답 선택 또는 후처리 단계에 결합하는 방식이 점차 확산되고 있다. 이에 Qwen3 계열 모델을 활용하여 Reader 단계에서의 확장 가능성을 두 가지 방향으로 실험하였다.

(1) Qwen3 기반 Answer Re-ranking (Refinement Reader)

첫 번째 실험에서는 Encoder 기반 Reader의 span 추출 능력을 유지한 채, Qwen3를 정답 후보 선택(refinement) 용도로 결합하는 구조를 적용하였다. Retriever를 통해 선별된 top-k 문서에 대해 Encoder Reader가 문서별 span 기반 추론을 수행하고, Reader

score(start/end logits 합)와 Retrieval score를 가중합한 통합 스코어(combined score) 기준으로 상위 N개의 정답 후보를 선정하였다. 이후 Qwen3(0.6B / 1.7B)에 질문, 관련 context, 후보 답변을 함께 입력하여 후보 중 가장 적합한 정답을 선택하도록 제한된 프롬프트를 사용하였다. 이 과정에서 새로운 답변 생성을 방지하고, 최종 출력이 실제 문서 내에 존재하는지를 검증하는 필터링을 적용하여 hallucination을 최소화하였다.

(2) Qwen3-4B 기반 Generative Reader 대체 실험

두 번째 실험에서는 Encoder Reader를 완전히 제거하고, Qwen/Qwen3-4B-Instruct-2507 모델을 Reader로 직접 활용하는 구조를 구성하였다. 전체 파이프라인은 기존 Retriever를 유지한 Two-stage 구조 ([Retriever] → [Qwen3 Reader] → [Postprocess])로 설계하였다. KorQuAD 스타일 extractive QA 데이터를 기반으로, context와 question을 입력받아 정답 span을 문서에서 그대로 복사해 출력하도록 유도하는 프롬프트를 구성하였으며, LoRA 방식으로 미세조정을 수행하였다. 추론 단계에서는 Top-1 또는 Top-k 문서를 context로 제공하고, 출력 결과에 대해 문자열 정규화 및 문서 내 존재 여부 검증을 포함한 후처리를 적용하였다.

(3) 실험 결과

Idea	Public		Private	
	EM	F1	EM	F1
Qwen3-based Answer Re-ranking	61.25	72.48	57.78	71.71
Qwen3-4B Generative Reader	61.25	71.77	57.50	69.74

전반적으로 두 실험 모두 F1 대비 EM이 상대적으로 낮은 경향을 보였다. 이는 LLM 기반 Reader가 정답 의미를 포착하는 데에는 강점을 보이지만, EM 기준에서 요구하는 완전 일치(span-level exact match)에는 불리한 출력 특성(불필요한 공백, 문장부호, 표현 변형 등)이 잔존했기 때문으로 해석할 수 있다. 특히 Generative Reader 설정에서는 후처리를 적용했음에도, span 단위 정합성 측면에서 기존 Encoder 기반 Reader 대비 안정성이 떨어지는 한계가 관찰되었다.

결과적으로 Qwen3 기반 Reader 확장은 성능의 유의미한 향상으로 이어지지는 않았으며, 실험 자원 및 시간 제약으로 인해 추가적인 프롬프트 설계나 학습 전략 개선 실험은 진행하지 못하였다.

2-5. 앙상블

개별 모델의 추론 결과를 합쳐 하나의 결과를 출력함으로써 추론 능력과 일반화 성능을 올리는 전략이다.

Model	Public		Private	
	EM	F1	EM	F1
Curriculum Learning based on Embedding Score	64.17	75.49	60.00	72.84
Fine-Tuned concat Embedding	63.75	75.52	58.06	69.80

Use Negative Passage	64.58	76.09	62.78	77.22
Curriculum Learning based on Negative Passage	65.00	77.10	66.67	78.38
uomnf97 learned Korquad v2	67.50	79.14	62.78	74.58

2-5-1. Hard Voting

모델들의 출력 중 가장 빈번하게 나온 답변을 선택합니다. 만약 동점일 경우, 베이스로 선정한 모델의 출력을 답변으로 선택한다.

2-5-2. Soft Voting

개별 모델 답변과 다른 모델들과의 유사도를 계산함으로써, 의미론적 유사성을 파악하여 가장 유사도가 높은 출력을 답변으로 선택한다.

Idea	Public		Private	
	EM	F1	EM	F1
Hard Voting	69.17	80.00	68.89	80.65
Soft Voting	71.25	80.87	68.89	80.60

3. 개인 회고

3-1. 최준호

1. 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?

대회 시작 전 개인적인 목표를 “기존의 해결방법에 대해 나만의 해석을 첨가하여 개선해보자”였다. 먼저 해당 Task에 대해 일반적인 해결법을 적용해보며 개선할 수 있는 부분을 찾았다. 그 결과 Reader 파트에서 Curriculum Learning과 Ensemble에 대해 최신 기술과 대회를 진행하며 얻은 인사이트를 추가해 성능을 개선하고자 하였다.

2. 나는 어떤 방식으로 모델을 개선했는가?

2-1. Curriculum Learning

기존 난이도 지표는 Passage의 특성 혹은 Answer과의 관계성을 비교하여 상대적인 순위를 매겼다. MRC Task에서는 Question에 대해 Context를 Cheating하여 답변을 출력하는 것으로 이 둘 사이 관련성을 반영하면 좋겠다 생각했고, 이 때 강의에서 다룬 내용인 단어의 존재 여부인 Sparse Embedding과 의미론적으로 분석하는 Dense Embedding을 활용하고자 하였다.

$$Embedding = \alpha * Sparse Embedding + (1 - \alpha) * Dense Embedding$$

alpha는 하이퍼파라미터로, 0.5 이상을 주어 MRC 특성 상 Sparse Embedding에 더 가중치를 두기 위해 활용하였다.

Dense\Sparse	Low	High
Low	Difficulty	Slightly Easy
High	Slightly Difficulty	Easy

$$Position = Start of Answer / Len of Context$$

다음으로 활용할 지표는 Context 내 Answer의 상대적 포지션이다. 실제로 Train data에 활용할 수 있는 Start of Answer index 지표가 있었고 이를 활용하였다. 상대적으로 Context의 전반부에 위치할수록 정답을 찾기 쉽다고 생각하였다.

$$Difficulty = \alpha * Embedding + (1 - \alpha) * Position$$

위 지표를 활용해 최종 난이도 랭킹을 세웠다. 메인 아이디어인 Embedding에 더 가중치를 두기 위해 alpha를 0.5 이상 주었다.

2-2. Embedding based on LLM

기존 Ensemble은 단어의 빈도나 의미적 유사성 등 Fixed Strategy로 의사결정을 하였다. 이 때, 비슷한 상황에서 어떻게 했는지에 대한 예제를 활용하면 Ensemble 성능이 올라가지 않을까에 대한 가정에서 출발하여 현 Computing Unit에서 동작할 수 있는 Qwen Local based LLM을 활용하여 Few-shot example을 주고 활용하여 의사결정을 할 수 있도록 구성하였다.

동작 과정

1. 현재 Question, Context에 대해 Similarity Search 기반 Top-K Few shot Example을 가져온다.

2. Example에 대해 Question, Context, Model들의 Answer들, Ground-Truth를 제공해주어 비슷한 상황에서 예제를 Cheating하여 답을 도출한다.

비록 시간 제약으로 인해 최종까지 구현하진 못했지만 Example Pool 생성과 Random Search까지는 구현하였으며, Question과 Context 사이 유사도 기반 Similarity Search를 도전 과제로 삼고있다.

3. 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?

Curriculum Learning은 기존에 비해 3-4% 성능 향상이 있었고, Ensemble의 하나의 Node로 활용하였다.

Ensemble based on LLM은 아이디어 자체는 참신했고, 활용 가능성이 높았으나 7일간의 Competition 특성 상 여기에 이를 이상 시간을 쏟아부은 것은 팀에 부하를 가할 수도 있다고 느꼈다. 인공지능을 학문적, 연구적 관점이 아닌 공학적 관점으로 봤을 때, 한정된 시간 내에서 리스크를 최소화하기 위한 최적화된 스케줄링에 대해 고민할 수 있었다.

4. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

“짧은 시간에 팀원과 협업하여 하나의 목표를 완수한다”라는 관점으로 대회를 봤을 때, 크게 도전 과제는 짧은 시간에 최선의 퍼포먼스를 내는 것과 팀원과 협업하여 시너지를 낸다고 볼 수 있다. 보다 도전적인 시도를 통해 리스크를 높인 점, 팀원과 공유보단 독립적인 작업에 가까웠다는 점이 아쉬웠다.

5. 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?

우선 왜 독립적인 작업에 가깝냐라는 질문에 대해 스케줄링의 부재가 컸다. 자유롭게 시도해보며 인사이트를 얻는 것은 창의적인 아이디어를 많이 얻을 수 있지만 그만큼 기반이 부족하였고 개개인의 개성이 더 돋보이게 되었다. 하나의 팀으로서 어느정도의 규제를 하기 위해서는 데드라인과 그 데드라인이 기대하는 목표가 있는 스케줄링의 필요성을 느꼈고 다음 대회에 시도해보고자 한다.

다음으로 데이터에 대한 인사이트의 필요성을 더욱 여실히 느꼈다. 이는 멘토님께서도 해주신 피드백으로, 엔지니어로서 내 모델의 성능을 올리기 위해서는 내가 가진 데이터에 대한 이해도를 올리는데 집중해보고자 한다.

3-2. 우혜진

1. 나는 어떤 방식으로 모델을 개선했는가?

이번 프로젝트에서 주로 Reader 중심의 성능 개선 실험을 맡아 진행했다.

기본 Reader 구조 위에서 Retrieval 환경과의 괴리를 줄이기 위해 Negative Passage를 활용한 학습을 도입했고, 이를 확장해 Curriculum Learning 방식으로 난이도를 점진적으로 높이는 실험을 시도했다. 또한 Qwen3를 활용한 LLM 기반 Reader 및 refinement 방향의 실험도 병행하며, 기존 extractive Reader가 놓치는 부분을 보완하고자 했다.

Negative Passage를 구성하는 과정에서는 단순히 오답 문단을 섞는 것이 아니라, 실제 Retrieval 단계에서 발생하는 상황과 최대한 유사하도록 의미적으로 유사하지만 정답을 포함하지 않는 passage를 중심으로 설계하려 했다. 더불어 원본 위키 문서가 아닌 chunk 단위로 분리된 passage 환경에서도 정답 span이 유지되는지를 확인하며 데이터 구성의 타당성을 점검했다.

Curriculum Learning에서는 난이도를 passage 개수 기준으로 정의해 실험을 진행했으나, 이 기준이 충분한지에 대해서도 지속적으로 고민하며 실험을 조정했다.

Retrieval 파트는 직접적으로 크게 수정하지는 않았다. 다만 BM25 기반 sparse retrieval, BGE 계열 dense retriever, hybrid retrieval, cross-encoder reranker 등 다양한 전략을 분석하고 팀원들의 실험 결과를 해석하는 역할을 수행하며, 전체 파이프라인 관점에서 Reader 개선 방향을 고민했다.

2. 내가 한 행동의 결과로 어떤 지점을 달성하고 어떠한 깨달음을 얻었는가?

가장 크게 얻은 깨달음은 Retrieval이 전체 성능의 상한을 결정한다는 사실이다. Reader 구조나 학습 전략을 아무리 바꿔도, 정답 문서가 Top-k에 포함되지 않은 샘플은 끝내 개선되지 않는 경우가 많았다. 이를 통해 Retrieval과 Reader는 분리된 단계가 아니라, 공동으로 최적화해야 하는 하나의 문제라는 인식을 갖게 됐다.

또한 Negative Passage나 Curriculum Learning과 같은 기법을 적용하면서, 단순히 모델을 “더 강하게” 만드는 것보다 실제 추론 환경을 얼마나 잘 모사하느냐가 성능에 직결된다는 점을 배웠다. 특히 Reader 학습에서 발생하는 학습-추론 간 불일치가 모델의 취약성으로 이어질 수 있다는 점을 실감했다.

개인적으로는 하나의 문제에 장기간 몰입해보는 경험 자체가 큰 수확이었다. 체력적으로는 힘들었지만, 단순히 실험을 반복하는 것이 아니라 문제를 구조적으로 이해하려고 시도하는 과정에서 연구적인 사고 방식에 한 발 더 가까워졌다고 느낀다.

3. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

가장 큰 한계는 데이터에 대한 이해 부족이었다. 성능이 나오지 않는 상황에서 왜 특정 문서가 retrieve되지 않는지, Retrieval 결과로 가져온 문서나 데이터 자체에 오류나 편향은 없는지, 파이프라인이 정말 의도한 대로 동작하고 있는지에 대해 충분히 집요하게 의심하지 못했다. 결과적으로 모델을 만지는 데에 비해 데이터와 결과를 검증하는 데에는 상대적으로 소홀했다.

또 하나의 아쉬움은 성능 개선이 바로 나타나지 않는 실험을 너무 빨리 포기했다는 점이다. 구현과 수치 개선에 집중하다 보니, 기대한 결과가 나오지 않으면 그 원인을 끝까지 파고들기보다 실험 자체를 중단해버린 경우가 많았다. 특히 Qwen3를 활용한 LLM 기반 Reader 실험은 프롬프트 설계나 학습 전략을 더 다양하게 시도해볼 여지가 있었음에도 충분히 탐색하지 못한 채 마무리된 점이 아쉽게 남는다.

또한 팀 차원에서 실험 방향, 우선순위, 코드 관리 방식, 기록 기준에 대한 합의가 초기에 충분히 정리되지 않아 후반으로 갈수록 비효율이 누적되었다. 같은 코드를 다시 해석하고, 이전 실험을 복기하는 데 예상보다 많은 시간을 쓰게 된 점 역시 한계로 느껴졌다.

4. 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇일까?

다음 프로젝트에서는 모델 개선 이전에 데이터와 실험 결과를 더 집요하게 검증하는 단계를 먼저 가져가고 싶다. 왜 이 문서가 선택됐는지, 왜 빠졌는지에 대한 분석을 구조적으로 정리하고, 이를 바탕으로 모델 설계를 진행하는 방식으로 접근하고자 한다.

단기적인 성능 지표에만 매몰되기보다, 새로운 방법론의 가능성을 일정 기간 끝까지 탐색해보는 실험 전략을 가져가고 싶다. 특히 LLM 기반 Reader나 refinement 접근처럼 비용이 크더라도 장기적으로 의미 있는 방향성에 대해서는 더 깊게 시도해볼 계획이다.

마지막으로, 재현 가능한 코드 구조, 실험 기록, 리팩토링을 프로젝트 초반부터 명확히 정리하고, 팀 차원의 규칙으로 합의하는 것을 가장 우선순위에 두고 싶다.

Worry is like a rocking chair, it keeps you moving but doesn't get you anywhere.
(걱정은 흔들의자와 같아서 계속 움직이게 만들지만, 아무 데도 데려다주지는 않는다.)

이번 프로젝트를 통해 느낀 많은 아쉬움들을 불필요한 걱정으로만 남기기보다는, 다음 시도를 위한 에너지로 바꿔가고 싶다. 너무 걱정하지 말자. 걱정은 대부분 생각보다 훨씬 소모적이니까!

3-3. 김윤희

1. 나의 학습 목표, 시도한 방법, 실패에 대한 고찰

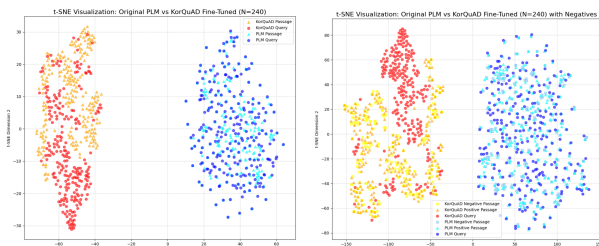
이번 ODQA 대회에서 설정했던 여러 학습 목표와 가설 중, 가장 아쉬움이 컸고 동시에 깊이 회고해보고자 하는 다음 가설에 대해 작성해보려고 한다.

Pretrained Language Model 을 dense retriever 로 사용하는 것보다, QA 데이터셋 (korquad2, KorNLI) 으로 fine-tuning을 진행했을 때 성능이 더 향상될 것이다.

이 가설을 검증하기 위해 KorQuAD2 데이터셋을 활용하여 jhgan/ko-sroberta-multitask 모델을 fine-tuning 하였다. Sentence-BERT 구조를 기반으로 query와 정답 passage를 positive pair로 구성하고, random negative sampling을 추가하여 contrastive learning 형태로 학습을 진행하였다.

이러한 접근을 선택한 이유는 다음과 같다. QA 데이터셋을 통해 query와 정답 passage 간의 의미적 alignment를 강화할 수 있다면, 그 결과 dense retriever가 질문에 대해 더 정확한 문서를 상위에 랭킹할 수 있을 것이라 기대했기 때문이다.

그러나 실제 실험 결과는 위 표와 같이 PLM보다 더 낮은 성능을 보였다. 대회 기간에는 이러한 성능 하락에 대해 깊이 고찰하기보다는, “이 방법은 아닌 것 같다”는 판단 하에 다른 모델이나 설정을 빠르게 시도하며 리더보드 성능에 집중하는 태도를 취했다. 비록 대회는 종료되었지만 이 결과를 구조적으로 이해해보는 과정이 필요하다고 느꼈다.



위는 query, positive passage, random negative passage 임베딩을 t-SNE로 시각화하여 PLM과 fine-tuned 모델의 embedding space를 비교했다. 그 결과 PLM의 경우 query와 passage가 완전히 겹쳐지는 않지만 전체 공간이 비교적 넓게 분산되어 있었으며, random negative passage가 query 주변으로 과도하게 밀집되는 현상은 관찰되지 않았다. 즉, query와 negative 간 최소한의 분리가 유지되고 있었다. 반면 fine-tuned 모델의 경우 query와 positive passage 뿐 아니라 negative passage 까지 query 주변으로 밀집되는 현상이 관찰되었다. 결과적으로 query-positive, query-negative 간의 구분이 매우 희미해졌으며 dense retrieval의 성능이 붕괴되었는지에 대한 구조적 설명을 제공해 주었다.

이러한 현상에 대해 다음과 같은 원인을 생각해볼 수 있었다.

첫째, KorQuAD dataset에 대한 특성이다. retrieval을 위한 ranking objective가 아니라, 질문-정답 쌍의 alignment를 강화하는 방향으로 임베딩 공간을 왜곡할 가능성이 있다고 생각한다. Context 역시 대부분 정답 span을 중심으로 구성되어 있어 질문과 어휘적, 구문적 유사성이 높은 문단들로 이루어져 있다. 이러한 데이터에 대해 cosine similarity 기반 학습을 진행할 경우, 모델은 “수많은 문서 중에서 정답 문단을 구별하는 능력” 보다는 “질문과 전반적으로 비슷한 문장을 끌어오는 능력”을 강화하게 될 가능성이 크다. 그렇다면 embedding space는 retrieval에 필요한 분별력을 잃고 query 중심으로 collapse 되는 방향으로 왜곡 될 수 있을 것이다.

둘째, negative passage를 충분히 멀리 밀어내는 학습 신호의

부족이다. Random negative sampling의 한계, 제한적인 batch size로 인해 효과적인 in-batch negative 학습이 이루어지지 못했고, 이는 negative separation을 약화시키는 요인으로 작용했을 가능성이 크다. 이러한 조건들이 결합되어 embedding space가 collapse에 가까운 형태를 띠게 되었고, 결과적으로 dense retrieval 성능 하락으로 이어졌다고 생각한다.

마지막으로 PLM 역시 이상적인 embedding space를 갖고 있다고 보기 어렵다고 생각한다. 시각화 결과에서 확인할 수 있듯이, PLM의 경우 fine-tuned 모델에 비해 query와 random negative passage 간의 분리는 유지되고 있었으나, query-positive passage 간의 alignment는 상대적으로 약하게 나타났다. 즉, PLM은 query와 무관한 문서를 효과적으로 제거하는 능력은 어느 정도 갖추고 있지만, 질문과 실제로 관련 있는 문서를 상위로 끌어올리는 데에는 한계가 있는 구조였다. jhgan/ko-sroberta-multitask(PLM) 모델은 일반적인 언어 모델보다는 문장 간 의미적 유사도를 잘 포착하도록 fine-tuning 된 Sentence-BERT 계열 모델이다. 그러나 이 역시 dense retrieval에서 요구되는 ranking objective나 명시적인 negative separation을 목표로 학습된 모델은 아니기 때문에, query-positive 간의 명확한 margin을 형성하는 데에는 한계가 있다.

2. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

1. 개인적 태도의 한계

대회 기간 동안 이러한 구조적 고찰을 충분히 진행하지 못했던 점이 가장 아쉬웠다. 모델 성능이 기대만큼 나오지 않았을 때, “왜 안 되는가?”를 깊이 파고들기보다는 다른 모델이나 설정을 빠르게 시도하며 리더보드 점수에 집중했다. 그 결과 문제의 본질을 이해하기보다는 증상에 대응하는 방식의 실험이 반복되었다.

2. 팀원 간 커뮤니케이션 방식

ODQA 시스템 전반에 대한 이해를 위해 초반에는 역할을 명확히 나누지 않고 진행했다. 이 접근은 시스템 이해 측면에서는 긍정적이었으나, 각자가 수행한 실험과 시도한 방법이 체계적으로 공유되지 못했다. 그 결과 유사한 실험이 반복되거나, 팀 전체가 정해진 상태에 머무는 비효율적인 순간들을 경험하게 되었다.

3. 컴퓨팅 자원의 제약

6명이 3개의 서버를 나누어 사용하는 환경에서 실험을 진행하다 보니, 동시에 무거운 모델을 학습하거나 가중치를 저장하는 데 제약이 많았다. 디스크 용량 초과로 인한 학습 중단, 가중치 저장 실패 등은 실험의 연속성과 다양성을 제한하는 요소로 작용하였다.

3. 다음 프로젝트에 시도해보고 싶은 점은 무엇인가?

1. 태도적 측면

다음 프로젝트에서는 모델 성능 개선 이전에, 가설과 목표가 문제 설정에 적합한지 먼저 점검하는 과정을 명확히 가져가고자 한다. 또한 코드 구현 단계에서 어떤 부분이 핵심이며, 실험이 잘 되지 않았을 경우 그 원인을 구조적으로 분석하는 습관을 유지할 것이다. 무엇보다도 “이 모델은 어떤 문제를 풀도록 학습되고 있는가?”라는 질문을 지속적으로 던지며 실험을 설계하고 해석하고자 한다.

2. 협업적 측면

팀원 간 실험 공유와 커뮤니케이션을 보다 체계적으로 관리하고, 역할 분담과 일정 관리가 명확한 협업 방식을 적용하고자 한다. 이를 통해 불필요한 중복을 줄이고, 정제되지 않고 지속적으로 발전하는 프로젝트를 진행하는 것이 목표이다

3-4. 객나영

1. 나는 내 학습목표 달성을 위해 무엇을 어떻게 했는가?

처음에 이 프로젝트를 시작하면서 설정한 목표는 “지난 개인 프로젝트보다 더 많은 실험과 더 원론적인 학습을 해보자” 그리고 “팀 차원에서 내가 수행한 실험들이 의미 있는 기여로 이어졌으면 좋겠다” 였다.

이전 개인 프로젝트에서는 주로 모델링과 리더보드 점수에 집중하여, k-fold validation과 앙상블 실험 위주로만 시도를 해보았다. 반면 이번 팀 프로젝트에서는 단순한 성능 개선을 넘어, 모델이 동작하는 구조와 개념을 보다 깊이 이해하며 학습해보고자 했다. 이를 위해 프로젝트 초반에는 Retrieval과 Reader 전반에 걸쳐 다양한 실험을 반복적으로 수행하는 것을 목표로 삼았다. 이후 실험을 진행하면서 전체 ODQA 성능에서 Retrieval 단계가 차지하는 영향이 크다는 점을 인식하게 되었고, 이에 따라 실험의 비중을 점차 Retrieval 구조 중심으로 옮겨갔다.

두 번째 목표를 위해서는 팀 내에서 모델링 관점의 실험이 주로 이루어지는 상황을 보고, 다른 영역에서 기여할 수 있는 지점을 찾고자 했다. 이에 따라 데이터와 Retrieval 구조 관점의 실험을 중심으로 실험을 진행하였고, 이 실험이 팀의 1차 Base Retrieval 구성으로 활용되면서 개인적으로 의미 있는 기여를 할 수 있었다.

2. 나는 어떤 방식으로 모델을 개선했는가?

모델 개선 과정에서는 Reader 모델의 성능 개선 보다는, Retriever 개선을 통해서 Reader가 입력으로 받는 문맥의 품질을 높이는 방향으로 접근하였다. Reader 모델 성능이 아무리 좋다고 해도 입력 데이터인 Context가 질문과의 관련성이 낮다면 ODQA의 전체 성능에 한계가 있을 것이라고 판단 했기 때문이다.

이에 따라 긴 문서 단위보다 passage 단위 문맥이 질문과의 관련성을 더 명확해질 수 있다고 판단하여, Wikipedia 문서를 passage 단위로 chunking하는 방식을 도입하였다.

또한 Sparse Retrieval(BM25)과 Dense Retrieval(bge-m3)을 결합한 Hybrid Retrieval 구조를 구성하고, 상위 후보 passage에 대해 Cross-Encoder 기반 Reranking을 적용함으로써 질문과 문맥 간의 정밀한 Relevance 판단이 가능하도록 개선하였다. 이를 통해 Reader 모델이 보다 정제된 문맥을 입력받을 수 있도록 하여 전체 성능 향상이 되도록 작업했다.

3. 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?

ODQA 성능 개선에 있어 Reader 모델 자체의 변경보다, Reader가 입력으로 받는 문맥의 품질을 결정하는 Retrieval 단계의 설계가 훨씬 큰 영향을 미칠 수 있다는 점을 경험적으로 확인할 수 있었다.

더 중요한 것은 개인적으로 수행한 실험과 결과를 팀원들과 보다 적극적으로 공유하고 소통하는 것의 중요성을 크게 느끼게 되었다. 팀 내에는 부스트캠프 8기 초기부터 함께 협업해온 팀원들이 있어서 이미 커뮤니케이션 흐름이 잘 형성되어 있었다. 그로 인해 의견을 적극적으로 제시하기보다는 팀에 끼지 못해서 괜히 혼자 눈치를 보게되는 분위기를 느끼게 되었고, 팀 논의에 적극적으로 참여하지 못한 점이 아쉬움으로 남았다.

그러나 팀 프로젝트를 진행하며 지켜보는 태도가 아닌 더 적극적으로 소통을 하고, 질문을 하고, 문제를 탐색하고 조사한 내용이나 실험 결과를 먼저 공유하는 태도가 중요하다는 점을 깨닫게 되었다. 이 경험을 통해 이후 협업 상황에서는 보다

주도적으로 의견을 제시하고, 팀에 기여할 수 있는 지점을 먼저 찾아 행동해야겠다는 깨달음을 얻었다.

4. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

이번 프로젝트는 팀으로 진행하는 첫 프로젝트이다 보니, 협업 측면에서 아쉬움이 많이 남았다. 팀원들 모두 개인 페이지에 실험 기록을 남기기는 했지만, 실험 결과를 팀 차원에서 충분히 공유하지는 못했다. 그로 인해 일부 실험은 몇몇 팀원만 알고 있는 상태로 남거나, 비슷한 실험이 팀 내에서 중복적으로 진행되는 경우도 있었다. 또한 현재 가장 성능이 좋은 모델이 무엇인지, 그리고 그 성능에 도달하기까지 어떤 과정을 거쳤는지가 팀 전체에 명확히 공유되지 못한 한계가 있었다.

개인적으로는 팀에 기여해야 한다는 생각이 강해지면서, 어느 순간부터는 팀원들과 협업하기보다는 팀 내에서 더 좋은 성능을 내야 한다는 경쟁적인 마음이 생기기도 했다. 지금 돌아보면, 그보다는 실험 결과를 더 적극적으로 공유하고 함께 방향을 맞추는 데 집중했어야 했다는 아쉬움이 남는다.

모델 측면에서는 Retrieval과 Reader를 각각 독립적인 단계로 개선하는 데 집중하다 보니, 두 모듈을 하나의 시스템으로 묶어서 함께 최적화하는 End-to-End 관점의 설계까지는 충분히 다루지 못했다. 시간 제한이 있고 리더보드 성능이 중요한 대회라는 특성상, 구조적인 이유로 모든 과정을 깊게 경험해보지 못한 점이 한계라고 느껴졌다.

특히 대회 상위 팀들의 결과를 살펴보면, 단순한 모델 구조 변경보다 데이터에 대한 분석과 이해가 성능 향상에 크게 기여할 수 있다는 점을 깨닫게 되었다. train/validation 데이터 분포를 다시 구성하거나, 질문-문서 간 유사성을 분석하는 등 데이터 관점의 접근을 충분히 시도하지 못한 점이 이번 프로젝트의 아쉬운 부분이었다.

5. 한계/교훈을 바탕으로 다음 프로젝트에서 시도해볼 것은 무엇인가?

이번 프로젝트를 통해 Retrieval 단계의 설계가 전체 MRC 성능에 미치는 영향이 매우 크다는 점을 분명히 인식하게 되었다. 다음 프로젝트에서는 Retriever와 Reader를 완전히 분리해서 보기보다는, 두 모듈이 서로 어떤 영향을 주는지까지 고려한 joint 학습 구조나 query reformulation과 같은 기법을 시도해보고 싶다. 이를 통해 전체 파이프라인을 하나의 시스템으로 바라보는 관점에서 실험을 진행해보고자 한다.

또 다음 프로젝트에서는 모델 실험 이전에 데이터 분석을 선행하는 것이 중요하다는 교훈을 얻었다.

또한 이번 프로젝트에서 실험 공유가 충분하지 못했던 경험을 바탕으로, 다음 프로젝트에서는 실험 결과를 팀 단위로 더 효과적으로 공유하는 방식을 적극적으로 적용해보고자 한다. 단순히 결과만 남기는 것이 아니라, 실험 목적-가설-결과-다음 액션을 명확히 구분하여 정리함으로써 팀원들이 현재 상황을 쉽게 이해하고, 이후 실험을 확장해 나갈 수 있도록 하고 싶다.

이를 위해 Jira와 같은 협업 도구를 활용하거나, 미팅을 더 자주 진행해서 각자가 어떤 작업을 진행하고 있는지, 그리고 그 결과를 바탕으로 함께 시도해볼 수 있는 다음 실험이 무엇인지 지속적으로 공유하는 구조를 만들어보고자 한다. 다음 프로젝트에서는 모델 성능뿐만 아니라 팀 협업 방식까지 함께 고민하며 프로젝트를 진행하는 것을 목표로 삼고 싶다.

3-5. 박서진

1. 나는 내 학습목표를 달성하기 위해 무엇을 어떻게 했는가?

본 프로젝트에서의 개인 학습 목표는 Open-Domain Question Answering(ODQA) 시스템의 전체 파이프라인을 이해하고, 특히 Retriever 단계에서 모델 선택과 실험 설계를 직접 경험해보는 것이었다. 이를 위해 Retriever 성능이 ODQA 시스템 전체 성능에 미치는 영향을 중점적으로 학습하고, 다양한 접근 방식을 비교 및 검토하는 데 초점을 두었다.

Retriever 중 Dense model을 선정하기 위해 multilingual-e5, Qwen embedding, BGE-M3 등 여러 Dense Retriever 후보 모델의 성능을 측정하였다. 동일한 데이터셋과 retrieval 설정(top-k 기준)에서 Recall@k 지표를 활용해 성능을 비교하였으며 이 과정에서 BGE-M3 모델이 가장 우수한 성능을 보임을 확인하였고, 이를 최종 Dense Retriever로 선정하는 데 기여하였다.

선정된 모델의 성능을 추가적으로 향상시키기 위해 질문-문서 간 relevance를 학습하는 방식으로 파인 튜닝을 시도하였다. positive-negative pair 구조의 학습 데이터를 재구성하고, cosine similarity 기반의 contrastive learning을 적용하여 모델 학습을 진행하였다.

또한 Sparse Retriever로 선정된 BM25의 성능을 추가적으로 향상시키기 위해 ElasticSearch를 활용한 실험을 시도하였다. 이를 통해 단순 키워드 기반 검색 방식에서도 설정 및 구조에 따라 성능 차이가 발생할 수 있음을 확인하고, Dense Retrieval과 Sparse Retrieval의 역할과 한계를 비교하며 Retriever 설계에 대한 이해를 넓히고자 하였다.

2. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

BGE-M3 모델을 파인튜닝하는 과정에서 여러 한계를 경험하였다. 초기 파인튜닝 실험에서는 positive passage와 negative passage로 구성된 공개 데이터셋을 활용하여 학습을 진행하였으나, 파인튜닝 이후 모델 성능은 오히려 크게 저하되었다. 이는 해당 데이터셋의 컨텍스트 길이와 질문 길이가 대회에서 제공된 데이터셋과 크게 달라, 실제 평가 환경과의 분포 차이가 발생했기 때문이라고 판단하였다.

이후에는 대회에서 제공된 데이터를 기반으로 contrastive learning에 적합한 데이터셋 구조로 재구성하여 파인튜닝을 다시 시도하였다. 이 경우 이전 실험과 달리 성능 저하는 발생하지 않았으나, 기대했던 Recall@k 성능 향상 또한 관찰되지 않았다. 이러한 결과는 negative sampling의 난이도와 데이터 구성 방식이 Retriever 학습에 충분한 학습 신호를 제공하지 못했기 때문이라고 생각한다.

더 근본적으로는 실험을 진행하기에 앞서 데이터 분포와 특성에 대한 충분한 분석을 수행하지 못한 점이 가장 큰 아쉬움으로 남는다. Dense Retriever 파인튜닝 과정에서 학습 데이터 구성과 negative sampling 전략을 체계적으로 설계하지 못했고, 그 결과 파인튜닝이 실제 Recall@k 성능 향상으로 이어지지 못했다. 이는 데이터 분석과 실험 설계가 모델 학습 성과에 직결된다는 점을 충분히 고려하지 못한 결과라고 생각한다.

아쉬웠던 점은 ElasticSearch 기반 Sparse Retrieval 역시 실험을 진행했으나, 대회 일정과 시간적 제약으로 인해 실제로 시스템에

적용하고 성능을 검증하는 단계까지 나아가지 못한 점이 가장 크게 아쉬웠다.

3. 한계와 교훈을 바탕으로 다음 프로젝트에서 시도해보고 싶은 점은 무엇인가?

이번 프로젝트를 통해 모델 선택이나 학습 기법에 앞서, 데이터 분석과 실험 설계가 선행되지 않을 경우 기대한 성능 향상으로 이어지기 어렵다는 한계를 명확히 체감하였다. 이를 통해 모델 자체의 성능도 중요하지만, 해당 모델이 학습하게 되는 데이터 구성과 실험 설계가 성능에 결정적인 영향을 미친다는 교훈을 얻었다. 이러한 한계와 교훈을 바탕으로, 다음 프로젝트에서는 실험에 앞서 데이터 분포, 질문-문서 길이, 정답 문서의 특성, hard negative의 난이도 등을 데이터를 체계적으로 분석하고, 이를 근거로 학습 데이터 구성과 실험 전략을 설계하고자 한다.

또한 프로젝트 초기 단계에서 태스크와 코드 구조를 보다 철저히 이해한 뒤, 실험 단위를 작게 나누어 빠르게 검증하고 결과를 해석하는 방식으로 실험을 진행할 계획이다. 이를 통해 단순히 많은 실험을 수행하는 데 그치지 않고, 각 실험의 목적과 결과를 명확히 연결할 수 있는 실험 흐름을 구축하는 것을 목표로 할 것이다.

이번 프로젝트는 아쉬움이 더 크게 남았지만, ODQA 시스템을 구성하는 각 단계의 역할과 한계를 명확히 인식하고, 다음 프로젝트에서 무엇을 개선해야 하는지를 구체적으로 정리할 수 있었다는 점에서 의미 있는 학습 경험이었다고 생각한다.

3-6. 김이슬

1. 나는 내 학습목표를 달성하기 위해 무엇을 어떻게 했는가?

이번 ODQA 프로젝트를 시작하며 설정한 나의 학습 목표는 베이스라인 코드의 전체 흐름을 이해하고, 이를 바탕으로 다양한 실험을 직접 설계하고 수행해보는 것이었다. 단순히 제공된 구조를 사용하는 데 그치지 않고, 각 단계가 어떤 역할을 하며 어떤 요소가 성능에 영향을 미치는지를 실제 실험을 통해 확인해보고자 했다.

이를 위해 먼저 Retrieval-Reader로 구성된 베이스라인 코드의 구조를 따라가며 전체 파이프라인을 이해했고, 이후 Retrieval 단계를 중심으로 실험을 진행하였다. 기존 QA 모델인 KorQuAD v1 데이터만 학습한 모델이라는 점에서 데이터 다양성이 부족하다고 판단하여 KorQuAD 2.0 데이터를 추가로 활용하였다. 다만 KorQuAD 2.0은 기존 데이터와 JSON 구조가 다르고, 위키피디아 기반의 HTML 태그·표 구조·편집 영역 등 노이즈가 포함되어 있어, 이를 그대로 사용할 수 없다고 판단했다. 이에 따라 데이터 구조를 변환하고 불필요한 노이즈를 제거하는 전처리 파이프라인을 직접 구현하여 실험에 활용하였다.

Retrieval 구조를 직접 다뤄보기 위해 Elasticsearch를 Docker 기반으로 외부 서버에 구축하고 애플리케이션과 통신하는 방식으로 검색 환경을 구성하였다. 한국어 검색 성능 개선을 위해 nori tokenizer, mixed decompound, shingle token filter를 적용하고, title 정보를 포함한 데이터 구성에서 Hybrid Retrieval 구조의 alpha 값과 rerank 후보 수를 조정하며 Recall@K 성능을 비교 실험하였다.

2. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

Retrieval 성능을 개선하기 위한 추가적인 시도로, LLM 기반 Multi-Query 실험과 질문(Question) 변형 실험도 함께 진행하였다.

Multi-Query 실험에서는 OpenAI의 o4-mini 모델을 활용해 하나의 질문으로부터 의미는 유지하되 표현이 다른 여러 질문을 생성하고, 이를 함께 사용해 검색 성능 향상을 시도하였다. 또한 Question 변형 실험으로는 질문에서 핵심 형태소를 추출하거나 질문 표현을 단순화하는 방식도 실험해보았다.

그러나 이러한 실험들은 기대했던 성능 향상으로 이어지지는 않았다. Multi-Query의 경우 질의 간 중복과 노이즈 증가로 인해 검색 결과의 안정성이 떨어졌고, Question 변형 실험 역시 문맥 정보가 손실되면서 일부 질문에서 성능이 오히려 하락하는 경향을 보였다.

3. 내가 한 행동의 결과로 어떤 지점을 달성하고, 어떤 깨달음을 얻었는가?

여러 실험을 시도했지만, 프로젝트 초반에는 데이터 자체를 충분히 분석하지 않은 상태에서 실험을 시작했다는 점이 가장 큰 아쉬움으로 남았다. 이후 마스터 클래스와 다른 조들의 발표를 보며, 데이터 분포 분석이나 질문 유형 분석을 바탕으로 실험을 설계한 사례들이 인상 깊게 다가왔다.

돌이켜보면, “이 설정을 바꾸면 성능이 좋아질 것 같다”는 직관에 의존한 실험이 많았고, 실험 결과가 좋지 않았을 때 그 원인을 데이터 관점에서 충분히 설명하지 못했다는 한계가 있었다. 특히

Multi-Query나 Question 변형 실험 역시 데이터 특성을 충분히 고려하지 못한 상태에서 진행되었다는 점이 아쉬움으로 남았다.

4. 마주한 한계는 무엇이며, 아쉬웠던 점은 무엇인가?

이번 프로젝트를 통해 베이스라인 코드를 깊이 이해하는 것이 모든 실험의 출발점이자 기준점이라는 사실을 체감하게 되었다. Elasticsearch를 직접 구축하고 설정을 변경해가며 실험한 경험을 통해, Retrieval 시스템이 내부적으로 어떤 방식으로 동작하는지에 대한 이해도 역시 크게 높아졌다.

또한 실험 결과가 최종적으로 채택되지 않더라도, 그 과정에서 “이 접근은 왜 효과적이지 않았는가”를 고민하는 경험 자체가 이후 실험 방향을 설정하는 데 중요한 기준이 된다는 점을 깨달았다. 성능 개선이 항상 수처로 드러나지는 않지만, 실패한 실험 역시 의미 있는 학습 과정이 될 수 있다는 점을 배울 수 있었다.

한편 이번 프로젝트에서 아쉬웠던 점 중 하나는 팀원 간 커뮤니케이션이 충분히 구조화되지 못했다는 점이다. 각자 실험 기록을 남기기는 했지만, 실험의 목적과 결과가 팀 전체에 명확히 공유되지 않으면서 유사하거나 중복된 실험이 반복되는 경우가 있었다. 그로 인해 시간과 자원을 보다 효율적으로 활용하지 못한 부분이 아쉽게 남는다.

개인적으로는 보다 적극적이고 능동적으로 프로젝트에 임하지 못했던 태도에 대한 아쉬움도 크다. 해야 할 일을 명확히 선언하고 주도적으로 가져가기보다는, 완성하지 못할 것에 대한 걱정으로 스스로 태스크를 크게 느끼며 한 발 물러서 있었던 순간들이 있었다. 지금 돌아보면, 결과의 완성도보다도 하나의 실험이라도 책임지고 수행하며 팀에 기여하는 경험을 더 중요하게 여기지 못했던 점이 아쉬움으로 남는다.

5. 한계와 교훈을 바탕으로 다음 프로젝트에서 시도해보고 싶은 점은 무엇인가?

다음 프로젝트에서는 모델이나 기법을 적용하기에 앞서 데이터 분석을 우선적으로 수행하는 접근을 시도해보고 싶다. 질문 유형, 문서 길이, 정답 분포 등 데이터의 특성을 충분히 파악한 뒤, 그 결과를 바탕으로 실험 가설을 설정하고 검증하는 방식으로 프로젝트를 진행하고자 한다.

또한 이번 프로젝트에서는 주로 Retrieval과 시스템 구성에 집중했다면, 다음에는 모델링 측면의 실험도 보다 적극적으로 시도해보고 싶다. 모델이 어떤 유형의 질문에 강점과 한계를 가지는지를 분석하며 실험을 설계함으로써, 베이스라인을 이해하는 단계에서 나아가 문제를 보다 주도적으로 정의하고 해결하는 경험을 쌓는 것이 목표이다.

아울러 이번 프로젝트를 통해 기술적인 실험뿐만 아니라 협업 과정과 작업 관리 방식 역시 프로젝트의 완성도에 큰 영향을 미친다는 점을 느끼게 되었다. 다음 프로젝트에서는 개인 작업을 단순히 코드로만 남기기보다, Git을 보다 적극적으로 활용해 PR(Pull Request) 단위로 작업을 공유하고 리뷰를 주고받는 협업 방식을 시도해보고 싶다.